

Taming XGBoost

How Regularisation and Tree Complexity Control Overfitting

Module	7PAM2021 — Machine Learning (Individual Tutorial, REF)
Technique	XGBoost (gradient-boosted trees) — focus: regularisation & the bias–variance trade-off
Dataset	UCI Concrete Compressive Strength (1,030 samples, 8 features)

Code & repository (GitHub link): <https://github.com/mani04180/xgboost-regularisation-tutorial>

1. Why this tutorial

XGBoost is fast, is able to deal with missing data, and is extremely accurate on the structured tabular data that is ubiquitous in real world databases, which is why it is able to take on the task of winning competitions and powering production systems. But there's a gem of a trap that sets in on almost every new user who needs to learn how to use it: XGBoost will be happy to memorise the training data and fail very hard on data it hasn't been exposed to – it's great in development, but not so great in production. This tutorial introduces a skill that is particularly important: the ability to detect overfitting as soon as it emerges and put it under control by modifying a small set of hyperparameters that control tree complexity and regularisation. You will be able to interpret and fine-tune a train-validation curve with XGBoost by the end.

2. A 60-second mental model

XGBoost is a realisation of gradient boosting. It constructs numerous small decision trees in sequence, and for each new tree it builds, it will only train it to correct the errors made by those already constructed. The final prediction is a composite of all trees with each tree's contribution reduced by a learning rate to ensure no single tree is able to dominate. It is the error correcting additive process that makes boosting so powerful — in fact, if it becomes unchecked, it can keep on adding trees until it fits every quirk and noise point in the training set.

As opposed to gradient boosting, XGBoost optimises prediction error but makes use of a lot more than just minimising error. Each time it splits it seeks to minimise a regularised objective, which penalises complexity:

$$\text{Objective} = \text{Loss}(\text{predictions}, \text{truth}) + \Omega(\text{trees})$$

The loss term quantifies the error of the predictions, and the penalty term quantifies the complexity of the model — the more leaves that a tree has, the smaller their associated weights, the larger the penalty term. (Chen & Guestrin, 2016) This penalty is already incorporated into the gain score for each candidate split and is thus automatically discounted if the split only fits noise. This second term is the essence of the tutorial: it's the tradeoff between the bias, a model that is too simple to capture the real pattern, and variance, a model so flexible that captures noise, which overfits. This dial is manipulated by 3 groups of parameters: tree depth, learning rate and explicit regularisation terms (reg_lambda, gamma and min_child_weight) together with random subsampling. Each is demonstrated in turn in this tutorial with an actual dataset.

3. The data and the diagnosis

We are asked to predict the compressive strength of concrete (in Mega Pascal, MPa) from the use of eight ingredients of the concrete (cement, slag, fly ash, water, superplasticiser, coarse aggregate, fine aggregate) and the age of the sample (in days). The 1030 rows do not contain any missing values and contain all numerical features, so we can focus on modelling. The selection of a small clean data set is intentional: it's small, it quickly overfits, it is clean, it's not one of the overly-used overly-popular tutorial datasets (Titanic, Boston, California housing). The data is divided into 60% training, 20% validation and 20% untouched test set. The validation set is our microscope during tuning and the test set is to be opened

only once, at the very end, to give an honest answer on how well the model would perform in the real world.

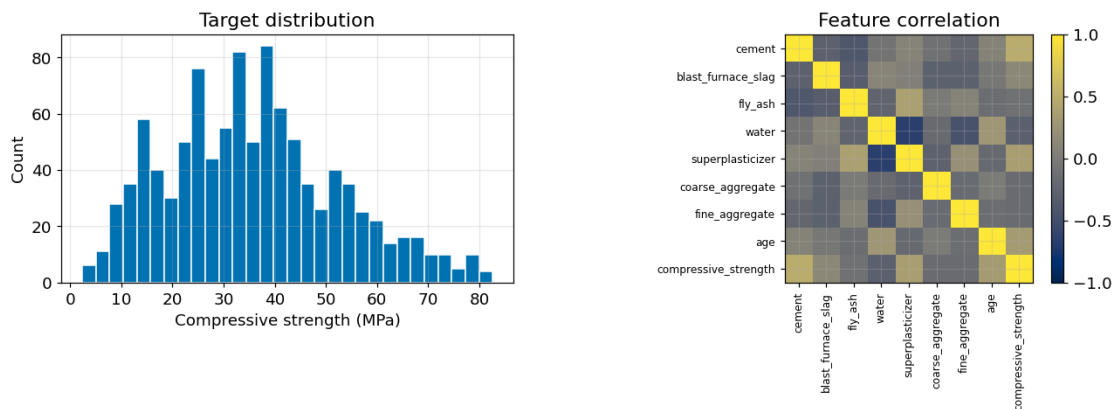


Figure 1. The target spans 2–83 MPa, and cement, age and water are the most strongly correlated ingredients.

To make the danger real, first train a model which is deliberately over-powered: very deep trees (12), no regularisation, high learning rate and hundreds of rounds. It is virtually perfect on the training data set (RMSE 0.96 MPa), but in the validation data set, it's a bit over five times out of the bag (RMSE 6.20 MPa). That's the hallmark of overfitting. All subsequent statements are about how to close it without discarding the true predictive property of the model.

4. Lever 1 — tree depth

`max_depth` restricts the number of consecutive questions that each tree can ask before it will have to make a prediction, and is the most powerful control over model complexity. The shallower are coarse, with high bias, and tend to underfit the feature space; the deeper are split into many small areas, with high variance, and tend to fit the feature space. When we sweep the depth from 1 to 12, we get the familiar-looking bias-variance graph in Figure 2, where the training error keeps decreasing, but the validation error has a minimum at a depth of 6 and then increases as the model begins to memorise. Here the bottom of that validation curve, at depth 4, is the "sweet spot": complex enough to "capture" the pattern, but not so complex that it captures any noise.

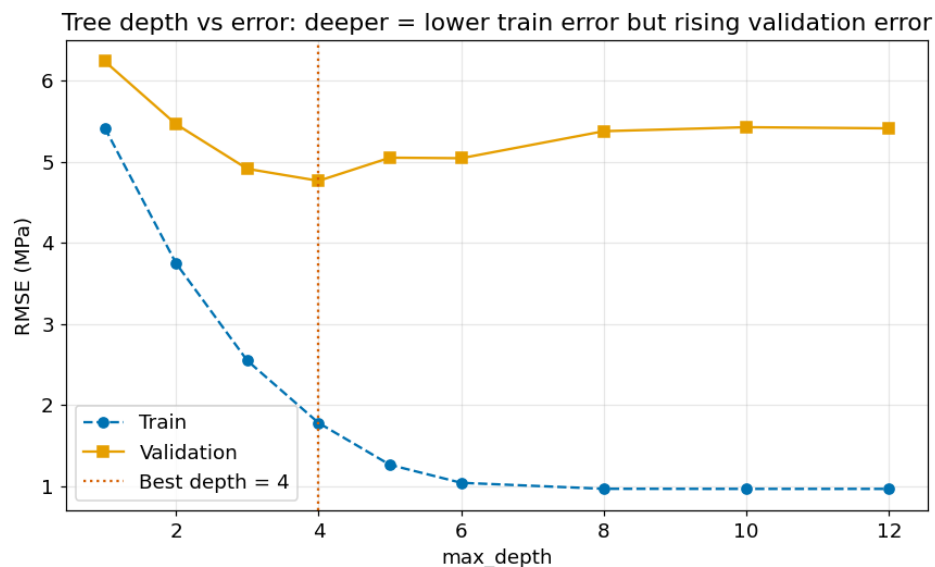


Figure 2. Validation error forms a U-curve: depth 4 balances bias and variance; deeper trees only memorise.

5. Lever 2 — learning rate

The learning rate (eta) reduces the contribution of each tree prior to the contribution being added to the running total. That size of the step at each round: a large step allows early trees to imprint their idiosyncrasies on the prediction, a small step means that early trees contribute a little, which averages out across lots of them. This means that a smaller rate is more generalising, but it is not free because it requires more trees to achieve the same accuracy - training time. The trade-off is clearly illustrated in Figure 3. The curves of the training and validation error are the lowest at high rates (0.3-0.5) which is the signature of over-fitting, and at small-to-moderate rates the curves of the training and validation error are the flattest and the lowest. Note that the 0.05 rate is a pretty solid starting point, but should be combined with plenty of boosting rounds.

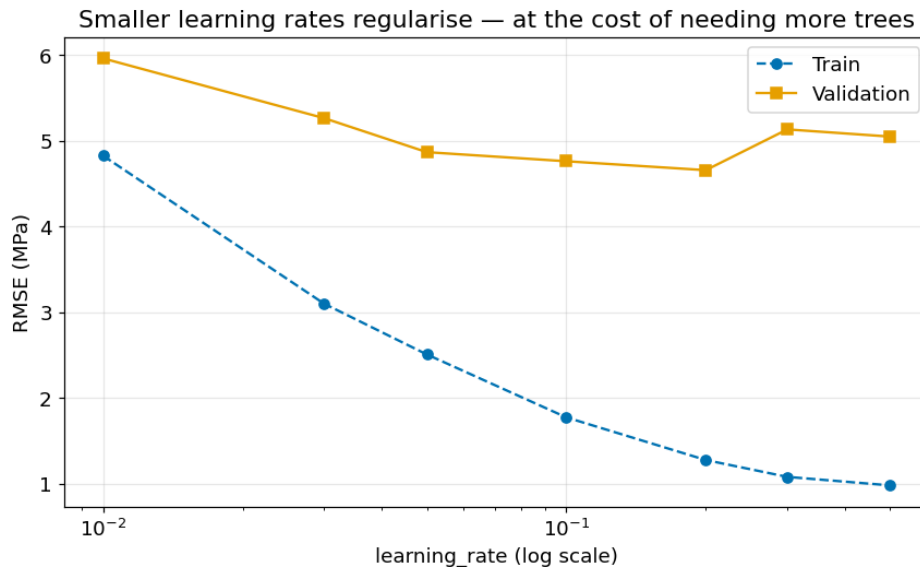


Figure 3. Lower learning rates regularise the model; the cost is needing more trees to converge.

6. Lever 3 — explicit regularisation

Past the whole depth and learning rate thing, XGBoost kind of gives us a way to punish complexity straight via the Ω term, with a bunch of knobs that each seem to fight overfitting from a different angle, not always in the same way. `reg_lambda` adds an L2 constraint on the leaf weights, so the predicted outputs get dragged toward zero and the booster can't just depend on a few extreme leaves. `gamma` instead controls the minimum gain a split must show, before it's even allowed to happen, so those "why bother" splits that are basically noise get chopped away. And `min_child_weight` makes sure every leaf has enough support in the data, meaning the model cannot slice tiny, jumpy leaves that don't really generalize. Figure 4 kinda isolates how `reg_lambda` behaves when the setup is already over-deep: when the penalty increases from 0 up to 100, the training error goes up a little, the train-validation gap gets smaller, and the validation error remains quite low. This is the tradeoff we're after exactly: we accept a bit less fit on the samples we already saw in exchange for a more credible accuracy on the data we haven't seen.

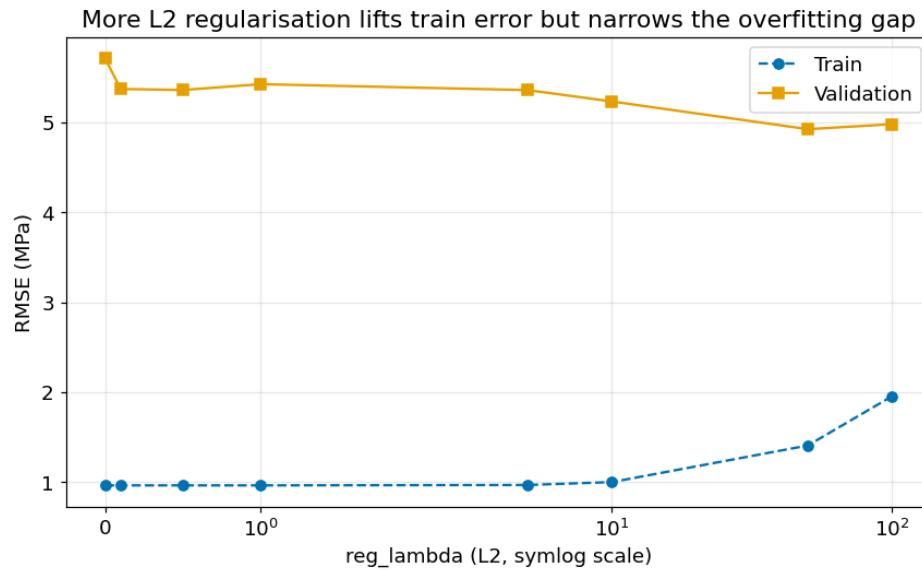


Figure 4. Stronger L2 regularisation raises training error but shrinks the overfitting gap.

7. Stop at the right time — early stopping

So yeah one question stays, how many trees should we actually build. Too few and everything underfits, too many and it starts drifting back into overfitting. Instead of guessing, early stopping sort of lets the data do the call. We hand the model a validation set, and also a patience budget (in this case 50 rounds); it keeps appending trees as long as the validation error keeps getting better and it will stop on its own once that error has not improved for that many rounds. When it stops, it also rolls back to the best iteration, not the very last one. Figure 5 shows both learning curves across the boosting rounds. Training error keeps dropping nonstop, which is kind of expected—training error can do that, it always will. The validation error however drops fast, then it basically levels off, and that is where you can see the beginning of overfitting forming. The moment the two curves start to split apart, that's the tell. Training ended by itself at round 439, capturing almost all the useful benefit without the bloat you'd get from thousands of pointless trees. This one tactic removes most of the guesswork around picking the number of estimators, and it adapts automatically if you change the learning rate or other setting.

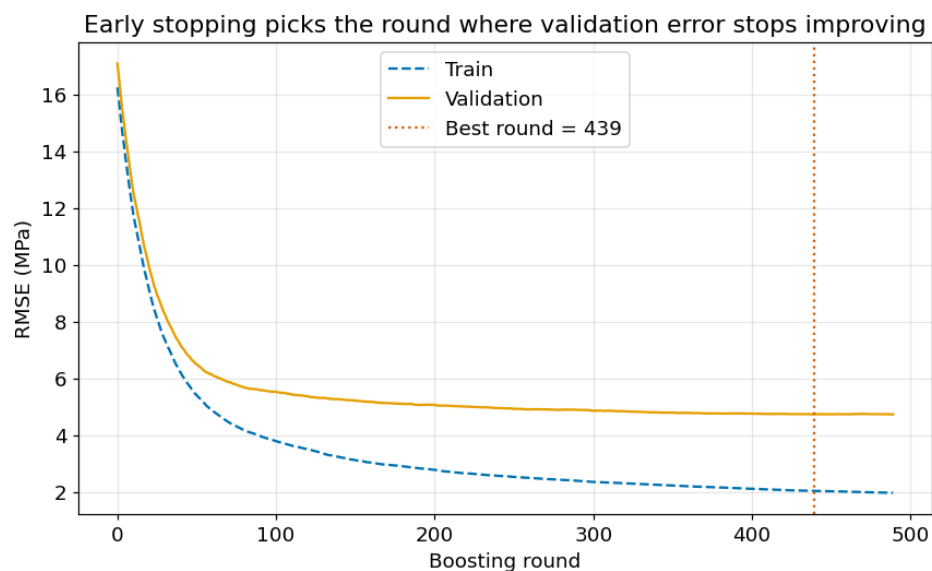


Figure 5. Training error always falls; early stopping picks the round where validation error plateaus.

8. The payoff

Now we open the test set, the data neither model has really seen before, and we compare the two: the original overfit model versus a tuned model. This tuned version uses a moderate depth of four, a 0.05 learning rate, L2 regularisation, 80 % row and column subsampling (kind of that randomised stance), plus early stopping. On the test set, the error goes from 6.47 down to 4.66 MPa, the test R^2 climbs from 0.84 to 0.92, and the train–test gap shrinks about half, from 3.26 to 1.71. So it's not only more accurate, but also more believable; the score it reports during development is much nearer to what you would get once it's deployed, not some overly optimistic guess.

Model	Train RMSE	Test RMSE	Train–Test gap	Test R^2
Overfit (deep, no regularisation)	3.21	6.47	3.26	0.838
Tuned (regularised + early stopping)	2.96	4.66	1.71	0.916

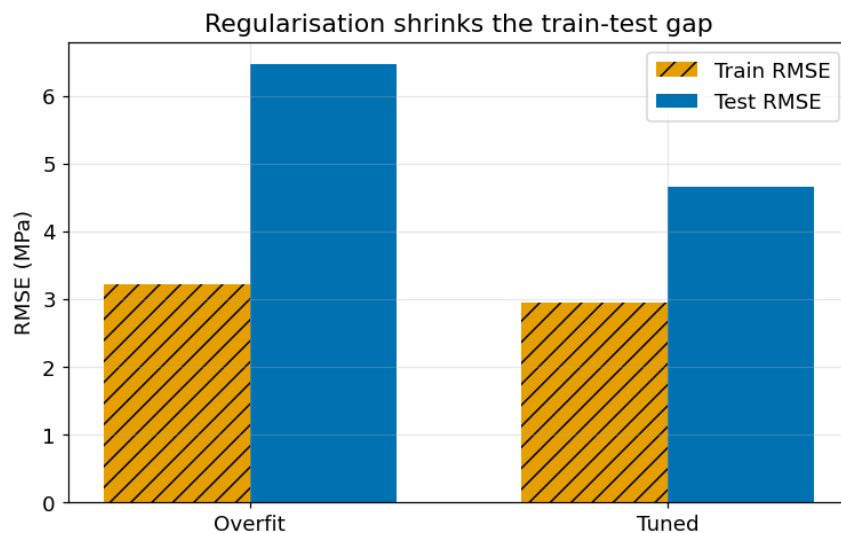


Figure 6. Regularisation narrows the train–test gap and improves generalisation on unseen data.

9. What the model learned

A regularised model is not only more accurate, it is also easier to trust because it leans on a sensible set of signals rather than noise, kinda. From a feature-importance perspective (Figure 7) measured by gain it looks like the tuned model weights age, cement and water the most, and that actually matches what is chemically sensible for concrete strength. Like, cement hydration over time is what gives the material its strength, and the water-to-cement ratio is a textbook determinant of that. So this quick sanity check matters: when the model's key features line up with domain knowledge, you tend to have more confidence that it learned something real. A small caution though—gain based importance is more of a rough map of where the model looks on average, not a strict claim about cause and effect. If you want explanations tied to each prediction, you would typically move on to SHAP values, which is a natural next step beyond this tutorial.

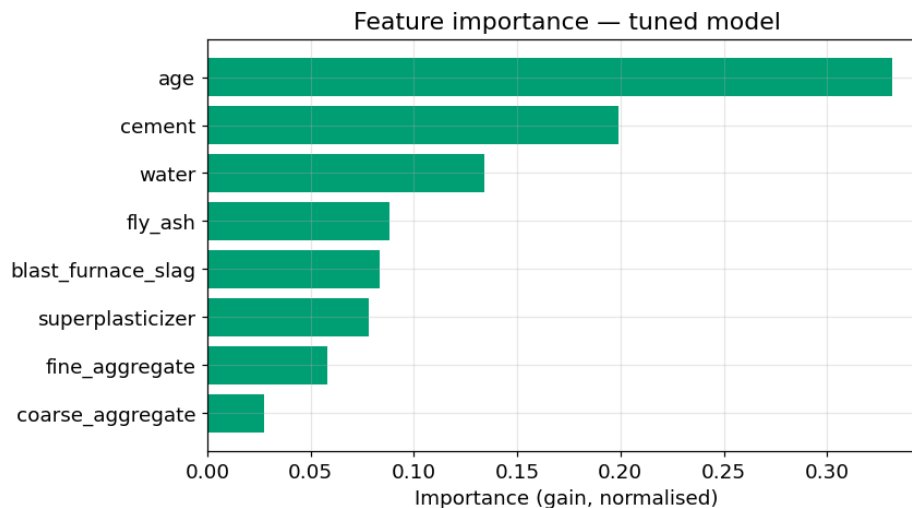


Figure 7. Age, cement and water dominate the tuned model's predictions.

10. A practical recipe

Pulling it together, a dependable workflow for your own tabular problems:

- Always track **train and validation together**; the gap between them is your single most useful overfitting gauge, far more informative than the training score alone.
- Start moderate rather than maximal: `max_depth 4-6`, `learning_rate 0.05`. Resist the urge to crank capacity up front.
- Add explicit regularisation and randomness — `reg_lambda`, `subsample` and `colsample_bytree` (around 0.8) — to reduce variance and decorrelate the trees.
- Let **early stopping** choose the number of trees on a validation set, so the count adapts whenever you change other settings.
- Tune with cross-validation for stability, and judge the final model *once* on a held-out test set so your reported score is honest.

11. Accessibility & reproducibility

Good teaching material should be usable by everyone, and honestly trustworthy by almost anyone. With that in mind, each figure in this tutorial uses the colour-blind-safe Okabe–Ito palette together with different line textures and little markers, so the plots stay readable even in greyscale and for people with colour-vision deficiencies. And yeah, each figure also includes descriptive alt-text, so screen-reader users are not left guessing.

The Jupyter notebook that comes along also fixes a random seed, it includes the dataset as a CSV, so it runs offline, and it should execute cleanly from top to bottom to recreate every single figure here. The intermediate steps are kept in, so the reasoning is visible.

All of this sits in the linked repository under the permissive MIT licence, so other folks can run it, tweak it, and keep building on it, without much friction.

References

1. Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD '16*. doi.org/10.1145/2939672.2939785
2. XGBoost developers. XGBoost Documentation — Parameters. xgboost.readthedocs.io/en/stable/parameter.html

3. Yeh, I-C. (1998). Modeling of strength of high-performance concrete using ANNs. *Cement and Concrete Research*. UCI dataset id 165: archive.ics.uci.edu/dataset/165
4. Okabe, M. & Ito, K. Color Universal Design (colour-blind-safe palette). jfly.uni-koeln.de/color